SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Archana J(192324294) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1: Doctor Shift Scheduling – Backtracking Search (CSP)

1. Problem Understanding

This is a Constraint Satisfaction Problem (CSP) with three components:

- **Variables:** D1, D2, D3 (the doctors).
- **Domain of each variable:** {Morning (M), Afternoon (A), Night (N)}, ordered M < A < N.
- **Constraints:**
- C1: D1 ≠ Night.
- C2: Shift(D2) < Shift(D3), i.e., D2 works before D3.
- C3: All-different, i.e., only one doctor per shift (no two doctors share a shift).
- C4: D3 ≠ Morning.
- C5: Every doctor gets exactly one shift (a complete assignment).

2. Method Selection

Backtracking Search (depth-first search that assigns one variable at a time and checks the constraints after each assignment). If a constraint is violated, the search abandons that branch (backtracks) and tries the next value. This prunes the search space: the full space has $3^3 = 27$ assignments, but backtracking checks far fewer.

Variable order: D1, D2, D3. Value order: Morning, Afternoon, Night.

3. Solution Process – Intermediate Steps and Constraint Checks

Step	Assignment tried	Constraint check	Result
1	D1 = Morning	C1 (D1 ≠ N) ✓; C3 ✓ (no other doctor assigned)	Accept → go deeper
2	D2 = Morning	C3 ✗ (Morning already used by D1)	Reject, try next value
3	D2 = Afternoon	C3 ✓; C1 not applicable ✓	Accept → go deeper
4	D3 = Morning	C4 ✗ (D3 cannot work Morning); C3 ✗	Reject
5	D3 = Afternoon	C3 ✗ (Afternoon already used by D2)	Reject
6	D3 = Night	C3 ✓; C4 ✓ (not Morning); C2 ✓ (D2 = Afternoon < D3 = Night); C1 ✓; C5 ✓ (all assigned)	**Accept – complete valid assignment**

Search stops at the first solution found. This run needed no backtracking out of D1 or D2, but the pruning at steps 2, 4 and 5 shows the constraint checks.

4. Exploring the Remaining Branches (for completeness and to verify the answer)

Branch	Assignment (D1, D2, D3)	Constraint violated / Result
D1 = M, D2 = A, D3 = N	(M, A, N)	Valid ✓

Branch	Assignment (D1, D2, D3)	Constraint violated / Result
D1 = M, D2 = N, D3 = A	(M, N, A)	C2 ✗ (D2 = Night is after D3 = Afternoon)
D1 = A, D2 = M, D3 = N	(A, M, N)	Valid ✓ (second solution)
D1 = A, D2 = N, D3 = M	(A, N, M)	C2 ✗ and C4 ✗
D1 = N, any	(N, –, –)	C1 ✗ – whole subtree pruned at once

5. Final Valid Schedule

Shift	Morning	Afternoon	Night
Doctor	**D1**	**D2**	**D3**

D1 → Morning, D2 → Afternoon, D3 → Night.

6. Verification

- C1: D1 works Morning, not Night ✓.
- C2: D2 (Afternoon) works before D3 (Night) ✓.
- C3: Each shift has exactly one doctor ✓.
- C4: D3 works Night, not Morning ✓.
- C5: All three doctors have exactly one shift ✓.

Note: a second valid schedule also exists (D1 → Afternoon, D2 → Morning, D3 → Night). With the stated variable and value ordering, backtracking returns the first one, (M, A, N).

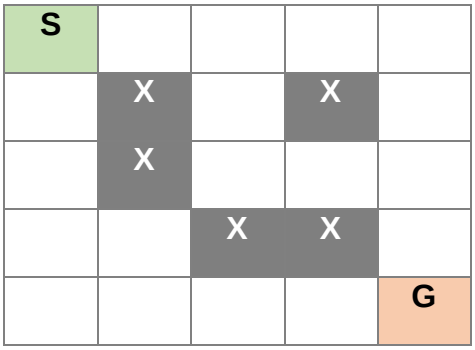
Question 2: Robot Path Planning in a 5×5 Grid (BFS / Manhattan Heuristic)

1. Problem Understanding

- 5 × 5 grid; the robot moves from **S** to **G** using Up, Down, Left, Right only (no diagonals).
- Each move costs 1; cells with obstacles cannot be entered; the heuristic is h(n) = |row − row_G| + |col − col_G| (Manhattan distance).
- Goal: find the optimal path and cost, and show node expansion and queue updates.

Note on the grid used: the grid figure (positions of S, G and obstacles) was not included in the text of the question paper. The solution below uses the following grid; the method and the table format are the same for any grid, so replace the cells with your paper's grid if it differs. Cells are written as (row, column), counted from the top-left corner as (1,1).

Assumed grid (S = Start (1,1), G = Goal (5,5), X = obstacle at (2,2), (2,4), (3,2), (4,3), (4,4)):



2. Method Selection

BFS expands nodes level by level using a **FIFO queue**. Because every move costs 1, BFS is complete and optimal: the first time it reaches G is via a shortest path. The question also asks for a priority queue with the Manhattan heuristic. That is the **A* algorithm**, which orders the queue by f(n) = g(n) + h(n) and gives the same optimal path with fewer expansions. Both are shown below.

Successors are generated in the order **Up, Down, Left, Right**; nodes already visited are not added again.

3. BFS – Step-by-Step Node Expansion (FIFO queue)

Step	Node expanded	New nodes added	Queue after expansion (front → back)
1	(1,1)	(2,1), (1,2)	(2,1) (1,2)
2	(2,1)	(3,1)	(1,2) (3,1)
3	(1,2)	(1,3)	(3,1) (1,3)
4	(3,1)	(4,1)	(1,3) (4,1)
5	(1,3)	(2,3), (1,4)	(4,1) (2,3) (1,4)
6	(4,1)	(5,1), (4,2)	(2,3) (1,4) (5,1) (4,2)

Step	Node expanded	New nodes added	Queue after expansion (front → back)
7	(2,3)	(3,3)	(1,4) (5,1) (4,2) (3,3)
8	(1,4)	(1,5)	(5,1) (4,2) (3,3) (1,5)
9	(5,1)	(5,2)	(4,2) (3,3) (1,5) (5,2)
10	(4,2)	–	(3,3) (1,5) (5,2)
11	(3,3)	(3,4)	(1,5) (5,2) (3,4)
12	(1,5)	(2,5)	(5,2) (3,4) (2,5)
13	(5,2)	(5,3)	(3,4) (2,5) (5,3)
14	(3,4)	(3,5)	(2,5) (5,3) (3,5)
15	(2,5)	–	(5,3) (3,5)
16	(5,3)	(5,4)	(3,5) (5,4)
17	(3,5)	(4,5)	(5,4) (4,5)
18	(5,4)	(5,5)	(4,5) (5,5)
19	(4,5)	–	(5,5)
20	(5,5)	Goal reached – stop	–

BFS expanded **20 nodes** (the goal is detected at the last step). The path is recovered by following parent pointers back from G to S.

4. A* with Manhattan Distance – Priority Queue Updates

g(n) = moves from S, h(n) = Manhattan distance to G(5,5), f(n) = g(n) + h(n). The priority queue always pops the smallest f; ties are broken by smaller h, then by insertion order. h(S) = |1−5| + |1−5| = 8, so f(S) = 8.

Step	Node popped	Successors added (g, h, f)	Priority queue after (node : f)
1	(1,1) (g=0,h=8,f=8)	(2,1)(g=1,h=7,f=8); (1,2)(g=1,h=7,f=8)	(2,1):8, (1,2):8
2	(2,1) (g=1,h=7,f=8)	(3,1)(g=2,h=6,f=8)	(3,1):8, (1,2):8
3	(3,1) (g=2,h=6,f=8)	(4,1)(g=3,h=5,f=8)	(4,1):8, (1,2):8
4	(4,1) (g=3,h=5,f=8)	(5,1)(g=4,h=4,f=8); (4,2)(g=4,h=4,f=8)	(5,1):8, (4,2):8, (1,2):8
5	(5,1) (g=4,h=4,f=8)	(5,2)(g=5,h=3,f=8)	(5,2):8, (4,2):8, (1,2):8
6	(5,2) (g=5,h=3,f=8)	(5,3)(g=6,h=2,f=8)	(5,3):8, (4,2):8, (1,2):8
7	(5,3) (g=6,h=2,f=8)	(5,4)(g=7,h=1,f=8)	(5,4):8, (4,2):8, (1,2):8

Step	Node popped	Successors added (g, h, f)	Priority queue after (node : f)
8	(5,4) (g=7,h=1,f=8)	(5,5)(g=8,h=0,f=8)	(5,5):8, (4,2):8, (1,2):8
9	(5,5) (g=8,h=0,f=8)	Goal reached – stop	–

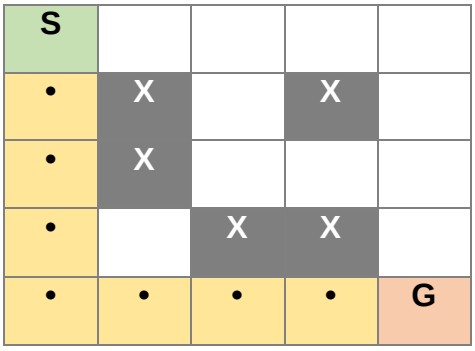
A* expanded only **9 nodes** compared with 20 for BFS, because the heuristic focuses the search toward G. Since h is admissible (it never overestimates, as every move costs at least 1) and consistent, A* is optimal.

5. Optimal Path and Cost

Optimal path: (1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5)

Moves: Down, Down, Down, Down, Right, Right, Right, Right.

Optimal cost = 8 (8 moves × cost 1 each).



6. Interpretation

- The cost 8 equals the Manhattan distance from S to G, which is the lower bound for any path in a 4-connected grid, so the path found is certainly optimal.
- BFS and A* give the same cost; A* is more efficient (9 vs 20 expansions). BFS ignores the heuristic and explores in all directions.
- Other paths of the same cost may exist; the tie-breaking order (Up, Down, Left, Right) decides which one is returned.

Question 3: Autonomous Rescue Robot – Online Search Under Partial Observability

1. Problem Understanding

A rescue robot must travel from S to the survivor at G in a building represented as a grid of rooms. It senses only adjacent cells, and the map is not fully known, so it must **plan, act, sense and update** while moving. It must minimise total cost while staying safe.

Cost model: a normal move costs 1; entering a risky zone costs 1 + 2 = **3**; obstacle (blocked) cells cannot be entered.

a) Analysis of the Constraints and Their Effect on Decision-Making

Constraint	Effect on decision-making
Moves: up, down, left, right	Small action set (branching factor ≤ 4); no diagonal shortcuts. Manhattan distance is a valid, admissible heuristic.
Blocked cells (obstacles)	Hard constraint: those cells are never in the successor function. The robot must find detours, and the plan may become invalid when a blockage is discovered.
Limited sensing (adjacent cells only)	The environment is partially observable; the robot cannot plan the full path in advance. It decides using only the known part of the map and must keep an internal model (memory).
Move cost 1, risky entry +2	Costs are non-uniform: the shortest path (fewest moves) is not always the cheapest. The robot must compare path cost, so BFS is unsuitable and a cost-based search is needed.
Minimise cost with safe navigation	Obstacles are hard constraints; risky zones are soft constraints with a penalty. The robot prefers safe cells and enters a risky zone only if it saves more than the penalty (a trade-off).
Dynamic knowledge update (online search)	The robot interleaves planning and acting; after each step it updates its map and re-plans if new information (a blocked path or risky zone) appears.

Priority of constraints: (1) blocked cells (hard, never violated) → (2) partial observability / online updating → (3) risk avoidance (soft, via cost) → (4) cost minimisation (objective).

b) Solution Approach – Online Search Agent with Uniform Cost Search / A* Replanning

Chosen strategy: an **online search agent** that uses **Uniform Cost Search (or A* with the Manhattan heuristic)** on its currently known map and replans whenever the map changes.

Steps involved:

- **Step 1 – Initialise:** put the robot at S; the known map is empty/unknown except S; set the goal G; cost of a move = 1; cost to enter a risky cell = 3.
- **Step 2 – Sense:** observe the four adjacent cells (free / blocked / risky) and add them to the knowledge base (map).

- **Step 3 – Plan:** run UCS (or A*) over the known map from the current cell to G, treating unknown cells as free (optimistic assumption) with cost 1. Use f(n) = g(n) + h(n) for A* with h = Manhattan distance, which is admissible since the minimum cost per move is 1.
- **Step 4 – Act:** execute only the **first step** of the best plan and move to that cell, updating total cost.
- **Step 5 – Update and replan:** sense again; if a planned cell is found to be blocked or risky, update the map and replan (go back to Step 3). Otherwise continue.
- **Step 6 – Terminate:** stop when the robot reaches G (survivor located). If no path exists in the known map and no unknown cells remain, report failure.

Pseudocode:

```
function ONLINE-RESCUE-AGENT(percept) returns an action
  persistent: map (known cells), position, plan, total_cost
  map ← UPDATE-MAP(map, percept)              // adjacent cells: free /
  blocked / risky
  if position = GOAL then return STOP
  if plan is empty or NEXT-CELL(plan) is blocked in map then
    plan ← UCS(map, position, GOAL)           // or A* with Manhattan
    heuristic
    // step cost = 1; entering a risky cell = 1 + 2; blocked cells
    excluded;
    // unknown cells assumed free with cost 1
    if plan = failure then return STOP        // no safe route known
  action ← FIRST(plan); plan ← REST(plan)
  total_cost ← total_cost + COST(action)
  return action
```

Small illustration of the cost trade-off: suppose route 1 from the robot to G has 4 moves and passes through one risky cell, cost = 4 + 2 = **6**; route 2 has 7 moves, all safe, cost = **7**. UCS/A* picks route 1 (6 < 7). If the penalty were larger (e.g. 5), route 1 would cost 9 and route 2 (7) would be chosen. A plain BFS would pick route 1 by move count without weighing the risk, and it cannot handle varying costs.

c) Application of AI Concepts

Intelligent agent

The robot is an **intelligent agent** that perceives its environment through **sensors** and acts through **actuators**. A PEAS description:

Performance measure	Environment	Actuators	Sensors
Survivor reached, minimum total cost, avoiding risky zones and collisions, time and energy	Disaster-hit building: grid of rooms, blocked paths, risky zones, survivor	Wheels/legs for up, down, left, right movement; rescue equipment	Adjacent-cell sensors (proximity/camera/thermal), position sensor, cost/battery monitor

Agent type: a **model-based, goal-based and utility-based agent**. It keeps an internal map (model) since the world is partially observable, it has the goal "reach G", and it uses cost (utility) to compare paths and risks.

Rationality

A rational agent selects, for each percept sequence, the action expected to maximise its performance measure given its built-in knowledge. Here, the robot is rational because it chooses the move that lies on the minimum-cost path known so far, avoids blocked cells, weighs the risk penalty against the path length, and replans when new percepts arrive. Rationality is not omniscience: the robot cannot foresee unobserved obstacles, but it makes the best decision from its current information, and it improves by gathering information (exploration).

Environment type

Property	Type	Reason
Observability	Partially observable	Sensing limited to adjacent cells.
Agents	Single agent	Only the robot acts (survivors/obstacles do not act).
Determinism	Deterministic (assumed)	A move leads to the next cell; (it could be stochastic if slipping/debris is considered).
Episodic / Sequential	Sequential	Present moves affect later cost and positions.
Static / Dynamic	Static (semi-dynamic if the building changes)	The map does not change while the robot decides, but new blockages may be discovered.
Discrete / Continuous	Discrete	Grid cells and unit moves.
Known / Unknown	Partly unknown	The layout is learned during exploration (online search).

d) Justification of the Chosen Approach

- **Online search is required:** because the robot sees only adjacent cells, a complete offline plan is impossible; online search interleaves planning and acting and adapts when the knowledge changes.
- **UCS / A* is required instead of BFS or DFS:** step costs are not uniform (1 vs 3), and BFS only minimises the number of moves, while DFS is neither optimal nor cost-aware. UCS is optimal for any positive costs; A* with the admissible Manhattan heuristic is optimal and expands fewer nodes.
- **Handles hard and soft constraints:** obstacles are removed from the state space (hard); risk is included in the path cost (soft), so safe routes are preferred automatically while still allowing a risky shortcut when it is clearly cheaper.
- **Replanning keeps the plan valid:** whenever new blocked or risky cells are discovered, the plan is recomputed, so the robot never follows an outdated plan.
- **Completeness and efficiency:** in a finite grid in which the goal is reachable, the agent will eventually find G (the cost of each replan is small). Alternatives such as LRTA* or D* Lite are also suitable online algorithms, but UCS/A* replanning is simpler and gives the optimal result on the known map.

Conclusion

An online search agent using Uniform Cost Search / A* with replanning meets all constraints: it respects blocked cells, works with limited sensing, accounts for the extra risk cost, updates its knowledge dynamically, and minimises the total cost of rescuing the survivor.